

CSE 214 HW02

Aditya Pahuja

1 Problem 1

If an element x in A has the property that $-(2x + 17)$ is also in A , then say that x is **valid**.

Algorithm 1(i) (FINDX-NAIVE($A[1 \dots n]$))

```
1. valuefound  $\leftarrow$  false
2. for  $i \leftarrow 1$  to  $n$  do
3.     for  $j \leftarrow 1$  to  $n$  do
4.         if  $A[j] = -(2 \cdot A[i] + 17)$  then
5.             print( $A[i]$ )
6.             valuefound  $\leftarrow$  true
7.             break
8. if not valuefound then
9.     print("no valid value found")
```

This algorithm is a brute force algorithm: for each $x := A[i]$, it checks each element of A to see if it is equal to $-(2x + 17)$. It then prints out the first such $A[i]$ and terminates, or, if no such $A[i]$ is found, prints that no valid value was found.

This is thus an $\mathcal{O}(n^2)$ algorithm, since in the worst case, if our desired value doesn't exist, the algorithm iterates through all pairs (i, j) with $1 \leq i, j \leq n$ (and processing such a pair takes $\Theta(1)$ time). Also, the extra space is $\Theta(1)$ because the only extra space used is for the valuefound variable.

Algorithm 1(ii) (FINDX-BETTER($A[1 \dots n]$))

```
1. valuefound  $\leftarrow$  false
2. create array  $B[1 \dots n]$ 
3. for  $i \leftarrow 1$  to  $n$  do
4.      $B[n + 1 - i] \leftarrow -(2 \cdot A[i] + 17)$ 
5. for  $i \leftarrow 1$  to  $n$  do
6.      $x \leftarrow A[i]$ 
7.     if BINARYSEARCH( $B[1 \dots n]$ ,  $-(2x + 17)$ )  $> 0$  then
8.         print( $x$ )
9.         valuefound  $\leftarrow$  true
10.    break
11. if not valuefound then
12.    print("no valid value found")
```

The way that B is defined is that the last (n th) element of B is $-(2 \cdot A[1] + 17)$, the second-to-last ($(n - 1)$ th) element is $-(2 \cdot A[2] + 17)$ and so on up to the first element $-(2 \cdot A[n] + 17)$ — in particular, B contains $-(2x + 17)$ for every x in A . Since A is sorted, the sequence $-(2 \cdot A[1] + 17)$, $-(2 \cdot A[2] + 17)$, $\dots$, $-(2 \cdot A[n] + 17)$ is decreasing ($x \mapsto -(2x + 17)$ is clearly decreasing in x), so B , whose elements are this sequence in reverse order, must be a sorted array.

What the algorithm does then is check whether any of the elements of A are also in B (via binary search on each element of A , since B is sorted!). This is equivalent to the existence of a valid value existing as, if some number y were in both A and B , then y being in B by definition means that $y = -(2x + 17)$ for some x in A , so x and $-(2x + 17)$ are both in A , hence x is a valid value. Conversely, if a valid x exists, then $-(2x + 17)$ is in A by the validity of x and in B by definition.

The time complexity of this algorithm is $\mathcal{O}(n \log n)$. Indeed, in the worst case where no valid value is found, we iterate through binary search n times, and each application of binary search takes $\mathcal{O}(\log n)$ operations because, by assumption, the binary search is being applied to the n -element array B in the worst case where it won't find the key. Additionally, FINDX-BETTER takes $\Theta(n)$ extra space due to generating the n -element array B .

Algorithm 1(iii) (FINDX-BEST($A[1 \dots n]$))

```

1. valuefound  $\leftarrow$  false
2. left  $\leftarrow$  1
3. right  $\leftarrow$   $n$ 
4. while right  $>$  0 and left  $\leq n$  and not valuefound do
5.     if  $A[\text{left}] = -(2 \cdot A[\text{right}] + 17)$  then
6.         print( $A[\text{right}]$ )
7.         valuefound  $\leftarrow$  true
8.         break
9.     else if  $A[\text{left}] > -(2 \cdot A[\text{right}] + 17)$  then
10.        right  $\leftarrow$  right  $-$  1
11.    else left  $\leftarrow$  left  $+$  1
12. if not valuefound then
13.    print("no valid value found")

```

To argue that this algorithm works we show that the algorithm will always print out the largest valid value (if it exists) in A . Clearly if no valid value exists then eventually either right will equal 0 or left will equal $n + 1$ and the algorithm will print “no valid value found.”

Now suppose that x is a valid value whose index in A is a (if x appears more than once in A , pick m to be as large as possible), and suppose that the index of the first occurrence of $-(2x + 17)$ in A is b . Eventually, right will equal a ; the algorithm can’t halt before this point because there are no indices larger than a corresponding to valid values. Then, left cannot be larger than b . To see this, suppose left were larger than b . left must have then previously equaled b , at which point right would be greater than a (if it equaled a then $A[a]$ would be detected as a valid value). Observe the inequality

$$A[b] = -(2 \cdot A[a] + 17) > -(2 \cdot A[c] + 17)$$

for all $c > a$, since $A[c]$ would be strictly greater than $A[a]$ (recall that a is the largest index giving value $A[a]$). Thus, when left = b and right $> a$, the algorithm would repeatedly decrease right by 1 until it equals a rather than increasing left. Now we’ve concluded that when right equals a , left is at most b . Symmetric to the above logic, if left is equal to b then we’re done; and, for every $c < b$,

$$A[c] < A[b] = -(2 \cdot A[a] + 17)$$

(recalling b is the smallest index giving value $A[b]$), so when left is smaller than b and right equals a , the algorithm keeps incrementing left until left equals b at which point $A[a]$ is found as a valid value.

The while loop takes at most $2n - 1$ steps to terminate, since right is allowed to be decremented at most $n - 1$ times and left is allowed to be incremented at most $n - 1$ times. The body of the while loop runs in constant time, which means that this algorithm is $\mathcal{O}(n)$. Furthermore, the only extra space is taken by the variables valuefound, left, and right, amounting to $\Theta(1)$ extra space.

2 Problem 2

If $A = [1, 2, 3, 4, 5]$, then these are the states the array passes through. Here, i is increasing top to bottom and j from left to right; the cell with index i is colored red while the cell with index j is colored green (except when $i = j$, which gets blue).

1 2 3 4 5 initial array				
1 2 3 4 5 (1,1)	2 1 3 4 5 (1,2)	3 1 2 4 5 (1,3)	4 1 2 3 5 (1,4)	5 1 2 3 4 (1,5)
1 5 2 3 4 (2,1)	1 5 2 3 4 (2,2)	1 5 2 3 4 (2,3)	1 5 2 3 4 (2,4)	1 5 2 3 4 (2,5)
1 5 2 3 4 (3,1)	1 2 5 3 4 (3,2)	1 2 5 3 4 (3,3)	1 2 5 3 4 (3,4)	1 2 5 3 4 (3,5)
1 2 5 3 4 (4,1)	1 2 5 3 4 (4,2)	1 2 3 5 4 (4,3)	1 2 3 5 4 (4,4)	1 2 3 5 4 (4,5)
1 2 3 5 4 (5,1)	1 2 3 5 4 (5,2)	1 2 3 5 4 (5,3)	1 2 3 4 5 (5,4)	1 2 3 4 5 (5,5)

If $A = [1, 2, 4, 3, 2]$:

1 2 4 3 2 initial array				
1 2 4 3 2 (1,1)	2 1 4 3 2 (1,2)	4 1 2 3 2 (1,3)	4 1 2 3 2 (1,4)	4 1 2 3 2 (1,5)
1 4 2 3 2 (2,1)	1 4 2 3 2 (2,2)	1 4 2 3 2 (2,3)	1 4 2 3 2 (2,4)	1 4 2 3 2 (2,5)
1 4 2 3 2 (3,1)	1 2 4 3 2 (3,2)	1 2 4 3 2 (3,3)	1 2 4 3 2 (3,4)	1 2 4 3 2 (3,5)
1 2 4 3 2 (4,1)	1 2 4 3 2 (4,2)	1 2 3 4 2 (4,3)	1 2 3 4 2 (4,4)	1 2 3 4 2 (4,5)
1 2 3 4 2 (5,1)	1 2 3 4 2 (5,2)	1 2 2 4 3 (5,3)	1 2 2 3 4 (5,4)	1 2 2 3 4 (5,5)

This algorithm always results in a sorted array. To prove this, we will prove the stronger statement that the k th iteration of the outer loop (i.e. after the pairs $(k, 1), (k, 2), \dots, (k, n)$ are all processed), the subarray $A[1 \dots k]$ is sorted and $A[k]$ is the maximal value in the array.

We shall prove this by induction, starting with the base case of $k = 1$. In this case, whenever j reaches the first index for which $A[j]$ is maximal, $A[1] < A[j]$ must hold, so the values at indices 1 and j swap; after this point there are no further swaps because there are no larger values than $A[j]$ in the array. Of course, the 1-element array $A[1 \dots 1]$ is trivially sorted.

Now suppose that $A[1 \dots k]$ is sorted and $A[k] = n$, and we are on the $(k+1)$ th iteration of the loop (i.e. $i = k+1$). “Weaken” the swapping condition to say $A[i]$ and $A[j]$ are swapped if $A[i] < A[j]$ or $A[i] = A[j]$ and $i > j$; obviously swapping the identical numbers doesn’t affect anything, so this is just a cosmetic change. Then, let $j \in \{1, \dots, k\}$ be the first index for which $A[k+1] \leq A[j]$. The first swap performed is between $A[k+1]$ and $A[j]$. If $j \leq k$, the new value $A[j]$ at index $k+1$ is now at most $A[j+1]$, so the next operation will be swapping the new $A[k+1]$ (which is $A[j]$) and

$A[j+1]$; then the new new $A[k+1]$ (equals to the initial $A[j+1]$) swaps with $A[j+2]$; and this will keep going until reaching $k+1$. The sequence of swaps thus results in the following situation:

$$\begin{array}{c|c|c|c|c|c} j & j+1 & j+2 & \dots & k & k+1 \\ \hline A[k+1] & A[j] & A[j+1] & \dots & A[k-1] & A[k] \end{array}$$

Here the array values $A[t]$ are the values of the array prior to all of the swaps listed out above. After this point no other swaps are made because $A[k]$ is maximal and the index is larger than $k+1$. By assumption $A[1] \leq A[2] \leq \dots \leq A[k+1] \leq A[j] \leq \dots \leq A[k]$ so the new $A[1 \dots (k+1)]$ is sorted and the new $A[k+1]$ (equal to $A[k]$ in the old array) is maximal.

3 Problem 3

Algorithm 3(i) (COUNTINVERSIONS-NONRECURSIVE($A[1 \dots n]$))

```

1. count  $\leftarrow$  0
2. for  $i \leftarrow 1$  to  $n$  do
3.   for  $j \leftarrow i+1$  to  $n$  do
4.     if  $A[i] > A[j]$  then count  $\leftarrow$  count + 1
5. return count

```

This algorithm iterates through all pairs of indices (i, j) with $1 \leq i < j \leq n$ and checks whether each pair is an inversion, incrementing the count whenever it finds one.

The algorithm has a time complexity of $\Theta(n^2)$ since it always processes $\binom{n}{2} = \frac{n^2-n}{2}$ pairs of indices, and it uses $\Theta(1)$ extra space for the count variable.

Algorithm 3(ii) (COUNTINVERSIONS-RECURSIVE($A[1 \dots n]$))

```

1. if  $n = 1$  then
2.   return 0
3. else
4.   terminalcontrib  $\leftarrow$  0
5.   for  $i \leftarrow 1$  to  $n-1$  do
6.     if  $A[i] > A[n]$  then terminalcontrib  $\leftarrow$  terminalcontrib + 1
7.   return COUNTINVERSIONS-RECURSIVE( $A[1 \dots (n-1)]$ ) + terminalcontrib

```

This recursive algorithm computes the number of inversions by counting the number of inversions among the first $n-1$ elements of A and adding this to the number of inversions formed by the terminal element; for the base case of $n=1$, there are of course trivially zero inversions.

The time complexity is

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ T(n-1) + \Theta(n) & n > 1 \end{cases}$$

which amounts to $T(n) \in \Theta(n^2)$, where the $+\Theta(n)$ comes from the for loop that computes terminalcontrib. The space complexity is

$$S(n) = \begin{cases} \mathcal{O}(1) & n = 1 \\ S(n-1) + \Theta(1) & n > 1 \end{cases}$$

with the $+\Theta(1)$ coming from keeping track of terminalcontrib, which means $S(n) \in \Theta(n)$.

4 Problem 4

Algorithm 4(i) (PREFIXSUM-NONRECURSIVE($A[1 \dots n]$))

1. create array $P[1 \dots n]$
2. $P[1] \leftarrow A[1]$
3. for $i \leftarrow 2$ to n do
4. $P[i] \leftarrow P[i-1] + A[i]$

This algorithm computes the prefix sums iteratively by initializing $P[1] = A[1]$ and using the identity

$$P[i-1] + A[i] = (A[1] + A[2] + \dots + A[i-1]) + A[i] = P[i].$$

The time complexity is $\Theta(n)$, since the loop runs n times and each run of the loop takes constant time. The space complexity is $\Theta(n)$, coming from the space taken up by the prefix array $P[1 \dots n]$.

Algorithm 4(ii) (PREFIXSUM-RECURSIVE($A[1 \dots n]$))

1. if $n = 1$ then
2. return $A[1 \dots 1]$
3. else
4. create array $P[1 \dots n]$
5. $P[1 \dots (n-1)] \leftarrow \text{PREFIXSUM-RECURSIVE}(A[1 \dots (n-1)])$
6. $P[n] \leftarrow P[n-1] + A[n]$
7. return $P[1 \dots n]$

This algorithm computes the prefix sums by first calling PREFIXSUM-RECURSIVE to compute the prefix sums for the first $n-1$ elements of the array, and then computes the n th prefix sum with the aforementioned identity $P[i-1] + A[i] = P[i]$.

The time complexity is

$$T(n) = \begin{cases} T(1) & n = 1 \\ T(n-1) + \Theta(1) & n > 1 \end{cases}$$

which implies that $T(n) \in \Theta(n)$. The space complexity is

$$S(n) = \begin{cases} S(1) & n = 1 \\ S(n-1) + \Theta(n) & n > 1 \end{cases}$$

where the $\Theta(n)$ comes from creating a new array every time PREFIXSUM-RECURSIVE is called, so the overall space complexity is $S(n) \in \Theta(n^2)$.

5 Problem 5

Algorithm 5(i) (ISVALIDSTRING($S[1 \dots 2n]$))

```

1. // returns true if  $S$  is valid and false otherwise
2. isvalid  $\leftarrow$  true
3. lcount  $\leftarrow$  0
4. rcount  $\leftarrow$  0
5. for  $i \leftarrow 1$  to  $2n$  do
6.     if  $S[i] = '('$  then lcount  $\leftarrow$  lcount + 1
7.     if  $S[i] = ')'$  then rcount  $\leftarrow$  rcount + 1
8.     if lcount < rcount then isvalid  $\leftarrow$  false
9. return isvalid

```

The main idea of this algorithm is that a string is valid if and only if, as one reads the string from left to right, the number of left parentheses (lcount) is always at least as large as the number of right parentheses (rcount). Clearly this is a necessary condition; to see that it is sufficient, one can use induction on n : the base case $n = 1$ is trivial (the only valid string is $()$), and in general if one deletes a matching pair of adjacent parentheses then one gets a string of length $2(n - 1)$ satisfying the same condition, which by hypothesis is valid and remains valid upon reintroduction of the deleted pair.

The time complexity here is $\Theta(n)$, as the loop runs exactly $2n$ times and the body of the loop runs in constant time. The space complexity is $\Theta(1)$ because the only extra space used is to store the variables isvalid, lcount, and rcount.

6 Problem 6

Algorithm 6(i) (GCD-NONRECURSIVE(a, b))

```

1.  $g \leftarrow 1$ 
2. for  $i \leftarrow 1$  to  $\min(a, b)$  do
3.     if  $a \% i = 0$  and  $b \% i = 0$  then
4.          $g \leftarrow i$ 
5. return  $g$ 

```

This algorithm just manually checks all numbers not larger than both a and b for whether they are common divisors of a and b .

The time complexity is $\Theta(\min(a, b))$ because the for loop always runs $\min(a, b)$ times (and the body of the for loop runs in constant time). The space complexity is $\Theta(1)$, with the only extra space used being g .

Algorithm 6(ii) (GCD-RECURSIVE(a, b))

```

1. for  $i \leftarrow 2$  to  $\min(a, b)$  do
2.     if  $a \% i = 0$  and  $b \% i = 0$  then
3.         return  $i \cdot \text{GCD-RECURSIVE}(a/i, b/i)$ 
4. return 1

```

This algorithm is a twist on the non-recursive edition: it checks for a divisor larger than 1 and, upon finding such a divisor d , computes $d \cdot \text{gcd}(a/d, b/d)$. However, this is not much of an improvement to the time complexity, since the case where $\text{gcd}(a, b) = 1$ still requires $\min(a, b)$ operations (and it's easy to see that the time complexity is bounded above by $\mathcal{O}(\min(a, b))$), so the algorithm is still $\mathcal{O}(\min(a, b))$.

In terms of space complexity the algorithm has a different worst case, namely when the number of recursive calls made is as large as possible. This necessitates that a and b share a lot of small prime factors, so the worst case is when $b = 2^n$ and b divides a , in which case the extra space is on the order of $\log n$, so in general the space complexity of the algorithm is $\mathcal{O}(\log n)$.

7 Problem 7

Algorithm 7(i) (EVALUATEPOLY-NAIVE($A[0 \dots n], x$))

```

1. value  $\leftarrow$  0
2. for  $i \leftarrow 0$  to  $n$  do
3.     powofx  $\leftarrow$  1
4.     for  $j \leftarrow 1$  to  $i$  do
5.         powofx  $\leftarrow$  powofx  $\cdot$   $x$ 
6.     value  $\leftarrow$  value + powofx  $\cdot$   $A[i]$ 
7. return value

```

This algorithm just computes each term of the polynomial by calculating x^i in $\mathcal{O}(i)$ fashion and then adds everything together. The time complexity is $\mathcal{O}(n^2)$ because of the nested for loops, and the space complexity is just $\Theta(1)$.

Algorithm 7(ii) (EVALUATEPOLY-BETTER($A[0 \dots n], x$))

```

1. value  $\leftarrow$  0
2. for  $i \leftarrow 0$  to  $n$  do
3.     powofx  $\leftarrow$  1
4.     exp  $\leftarrow$   $i$ 
5.      $y \leftarrow x$ 
6.     while exp  $>$  0 do
7.         if exp%2 = 1 then powofx  $\leftarrow$  powofx  $\cdot$   $y$ 
8.          $y \leftarrow y \cdot y$ 
9.         exp  $\leftarrow$  exp/2
10.    value  $\leftarrow$  value + powofx  $\cdot$   $A[i]$ 
11. return value

```

This algorithm runs almost identically to the previous one, but with an improvement on the computation of x^i by using binary exponentiation instead of linear exponentiation. The binary exponentiation improves the complexity to $\mathcal{O}(n \log n)$, and the space complexity remains as $\Theta(1)$.

Algorithm 7(iii) (EVALUATEPOLY-BEST($A[0 \dots n], x$))

```

1. value  $\leftarrow$  0
2. powofx  $\leftarrow$  1
3. for  $i \leftarrow 0$  to  $n$  do
4.     value  $\leftarrow$  value + powofx  $\cdot$   $A[i]$ 
5.     powofx  $\leftarrow$  powofx  $\cdot$   $x$ 
6. return value

```

This algorithm directly improves on the naive one by not throwing away power of x calculated in the preceding iteration, so that in a given run of the loop x^i can be computed as $x \cdot x^{i-1}$ where we've held on to the value of x^{i-1} from earlier. This makes the algorithm $\mathcal{O}(n)$ now, since the body of the for loop now runs in constant time. The space complexity is still $\Theta(1)$, like the two previous iterations of this algorithm.

8 Problem 8

Algorithm 8(i) (PAIRSUM-NAIVE($A[1 \dots n], k$))

```

1. // returns  $(i, j)$  if  $i < j$  and  $A[i] + A[j] = k$ , or  $(-1, -1)$  if no such  $(i, j)$  exists
2. for  $i \leftarrow 1$  to  $n$  do
3.     for  $j \leftarrow i + 1$  to  $n$  do
4.         if  $A[i] + A[j] = k$  return  $(i, j)$ 
5. return  $(-1, -1)$ 

```

This is a brute force algorithm that checks each pair (i, j) with $1 \leq i < j \leq n$ to see if $A[i] + A[j] = k$; there are $\binom{n}{2} \in \mathcal{O}(n^2)$ such pairs and each pair is processed in constant time, so the time complexity is $\mathcal{O}(n^2)$. The space complexity is $\Theta(1)$; nothing interesting happening here.

Algorithm 8(ii) (PAIRSUM-BETTER($A[1 \dots n], k$))

```

1. // returns  $(i, j)$  if  $i < j$  and  $A[i] + A[j] = k$ , or  $(-1, -1)$  if no such  $(i, j)$  exists
2. create array  $B[1 \dots n]$ 
3. for  $i \leftarrow 1$  to  $n$  do
4.      $B[n + 1 - i] = k - A[i]$ 
5. for  $i \leftarrow 1$  to  $n$  do
6.      $j \leftarrow n + 1 - \text{BINARYSEARCH}(B[1 \dots (n - i)], A[i])$ 
7.     if  $j > 0$  then return  $(i, j)$ 
8. return  $(-1, -1)$ 

```

The idea here is that for each $A[i]$, we are searching among all the elements appearing after $A[i]$ to look for $k - A[i]$; the way this is done mechanically is via the array B , which stores all the $k - A[i]$ s in reverse order (reverse to ensure that $B[1 \dots n]$ is sorted instead of reverse-sorted).

The time complexity of this algorithm is $\mathcal{O}(n \log n)$ due to the binary search being called n times; this asymptotic bound is sharp in the worst case where there is no (i, j) pair with $i < j$ and $A[i] + A[j] = k$. The space complexity is $\Theta(n)$ because the algorithm requires the production of the auxiliary array B with length n .

Algorithm 8(iii) (PAIRSUM-BEST($A[1 \dots n], k$))

```

1. // returns  $(i, j)$  if  $i < j$  and  $A[i] + A[j] = k$ , or  $(-1, -1)$  if no such  $(i, j)$  exists
2.  $\ell \leftarrow 1$ 
3.  $r \leftarrow n$ 
4. while  $\ell < r$  do
5.     if  $A[\ell] + A[r] = k$  then return  $(\ell, r)$ 
6.     else if  $A[\ell] + A[r] > k$  then  $r \leftarrow r - 1$ 
7.     else  $\ell \leftarrow \ell + 1$ 
8. return  $(-1, -1)$ 

```

This algorithm is in very similar spirit to the solution to Problem 1(iii). The idea is that increasing ℓ will increase the sum $A[\ell] + A[r]$, while decreasing r will decrease the sum because of the sorted nature of A . If an (i, j) pair with the desired properties doesn't exist, then it's clear that the algorithm will work because eventually ℓ will exceed r (after at most n steps, since $r - \ell$ strictly decreases after each iteration of the loop).

Otherwise, suppose that an (i, j) pair exists, and take i to be the smallest i appearing among all these pairs, so that j can be taken as the largest j among all these pairs (indeed, if there were a pair (i', j') with $j' > j$ then either $A[j'] = A[j]$, so we can just replace $j \leftarrow j'$, or $A[j'] > A[j]$ in which case $A[i'] < A[i]$ implies $i' < i$, contradiction). Then, by a similar argument to the one made in Problem 1(iii) (here with $k - x$ instead of the $-(2x + 17)$ expression), we can argue that eventually either ℓ will equal i or r will equal j , and then in the first case we must have r repeatedly decrease until it equals j , or in the second case we must have ℓ repeatedly incremented until it equals i , so the algorithm stops once it finds the valid pair $(\ell, r) = (i, j)$.

The time complexity is $\mathcal{O}(n)$, since the while loop runs at most n times, as remarked in the first paragraph, and the body of the while loop runs in constant time. The space complexity is $\Theta(1)$, as there are only some extra auxiliary variables (ℓ and r).

9 Problem 9

Algorithm 9(i) (COUNTFREQUENCY-NAIVE($A[1 \dots n], k$))

```

1. count  $\leftarrow 0$ 
2. for  $i \leftarrow 1$  to  $n$  do
3.   if  $A[i] = k$  then count  $\leftarrow$  count + 1
4. return count

```

This just checks every element of A for whether it equals k and increments the count when appropriate. The time complexity is clearly $\mathcal{O}(n)$ and the space complexity clearly $\Theta(1)$.

Algorithm 9(ii) (COUNTFREQUENCY-BEST($A[1 \dots n], k$))

```

1. first  $\leftarrow 0$ ; last  $\leftarrow -1$ 
2. if  $A[1] = k$  then first  $\leftarrow 1$ 
3. else
4.    $\ell \leftarrow 1$ ;  $r \leftarrow n$ 
5.   while  $\ell \leq r$  do
6.      $m \leftarrow (\ell + r)/2$ 
7.     if  $A[m] = k$  and  $A[m-1] \neq k$  then
8.       first  $\leftarrow m$ 
9.       break
10.    else if  $A[m] < k$  then  $\ell \leftarrow m + 1$ 
11.    else  $r \leftarrow m - 1$ 
12. if  $A[n] = k$  then last  $\leftarrow n$ 
13. else
14.    $\ell \leftarrow 1$ ;  $r \leftarrow n$ 
15.   while  $\ell \leq r$  do
16.      $m \leftarrow (\ell + r)/2$ 
17.     if  $A[m] = k$  and  $A[m+1] \neq k$  then
18.       last  $\leftarrow m$ 
19.       break
20.     else if  $A[m] > k$  then  $r \leftarrow m - 1$ 
21.     else  $\ell \leftarrow m + 1$ 
22. return last - first + 1

```

This algorithm finds the first instance of k in the array and the last instance of k in the array, and then uses these indices to compute the number of times k appears (since every number between the first and last k is equal to k because A is sorted).

The time complexity of this algorithm is $\mathcal{O}(\log n)$ because in both while loops, $r - \ell$ gets halved in each iteration of the loop, so the loops run $\mathcal{O}(\log n)$ times with constant time each iteration. The space complexity is obviously $\Theta(1)$.

10 Problem 10

To implement stacks using queues, we will treat the back of the queue as the top of the stack.

Algorithm 10(i) (PUSHWITHQUEUE)

1. // given queue Q behaving like a stack, push e to the stack
2. $Q.enqueue(e)$

Since the back of the queue is the top of the stack, adding to the top means just enqueueing e . Constant time, constant space.

Algorithm 10(i) (POPWITHQUEUE)

1. // given queue Q behaving like a stack, pop the top of the stack
2. if $Q.size() = 0$ then return null
3. create queue R
4. for $i \leftarrow 1$ to $Q.size() - 1$ do
5. $R.enqueue(Q.dequeue())$
6. popped $\leftarrow Q.dequeue()$
7. $Q \leftarrow R$
8. return popped

Here, we need to remove the element at the back of Q and return it. To do this we must dequeue $Q.size()$ times, the latter of which will be the popped value that we need to return. The remaining values are kept in a second queue R , which, because of how queues work, has its elements in the same order that they appeared in Q . Therefore we can simply relabel R as Q and then return the popped value. This is linear in both time and space complexity, owing to the creation of the extra queue.

For stacks, we will treat the top of the stack as the back of the queue and the bottom of the stack as the tail.

Algorithm 10(ii) (ENQUEUEWITHSTACK)

1. // given stack S behaving like a queue, enqueue e to the queue
2. $S.push(e)$

Symmetric to pushing with the queue, in order to enqueue e we just push e to the top of the stack (which is the back of the queue). Again, constant time and space.

Algorithm 10(ii) (DEQUEUEWITHSTACK)

1. // given stack S behaving like a queue, dequeue the front of the queue
2. if $S.size() = 0$ then return null
3. create stack T
4. for $i \leftarrow 1$ to $S.size() - 1$ do
5. $T.push(S.pop())$
6. dequeued $\leftarrow S.pop()$
7. for $i \leftarrow 1$ to $T.size()$ do
8. $S.push(T.pop())$
9. return dequeued

Here we have to do similar shenanigans with making a copy of the stack and popping every element to dequeue the one at the bottom. However, in this case, the storage unit T , being a stack, has its elements in the opposite order that they appeared in S (so the top of T was the second element from the bottom of S , while the bottom of T was the top of S). Thus we need to move the elements of T back into S as well. This again ends up having linear time and space complexities.